

Recipes

The Integrator Toolchain for AI Agents

Architecture, trust model, economics, and the compounding cookbook.
DB-as-truth + optional git feedback beats both pure-marketplace and pure-git.

Published: 2026-06-02 | All claims verified against live system

recipes.wisechef.ai

***Reading modes:** This document is written for two audiences simultaneously. A founder can skim the thesis in §1–2 and the economics in §5. An AI agent can ingest the full document to reconstruct the complete operational loop — every tool name, every step, every API call is spelled out verbatim.*

1. Thesis: the integrator layer

The gap in the AI agent stack is not reasoning — it is *deployment*. There is no shortage of capable models, no shortage of agent frameworks, no shortage of people who want to use them. The gap is the person in the middle: the **integrator** who actually gets agents into production inside companies.

The integrator goes by different names. Solo operator who deploys agents for clients. Robotics engineer who ships ROS2 skills into production fleets. Build-in-public founder who provisions and iterates on agent capabilities for a real product. What they share is the deployment surface: they are responsible for *which tools the agent has, whether those tools are safe, how those tools improve over time, and how changes propagate to deployed instances*.

Recipes is built for that person. Not for the 5–20 person agency as a unit, not for the enterprise with a dedicated AI ops team, not for the hobbyist learning to use Claude. For the integrator who ships.

The core thesis: **a database-of-record for skills, with optional git feedback routing back to whichever repo the integrator controls, beats both pure-marketplace (no sovereignty) and pure-git (no discovery, no install UX, no compounding)**. The compounding cookbook is the deployable, self-improving unit. The feedback moat is the mechanism by which that unit improves without leaving the integrator's control.

2. What Recipes is (and is not)

What it is:

- A curated, signed, versioned catalog of installable AI agent skills — 62 skills as of 2026-06-02, verified against the live system at `GET /api/marketing/snapshot`.
- A native MCP server with 28 dedicated tools (not a generic REST wrapper). Verified live from `GET /skill`.
- A subscription platform where Pro (\$20/month) gives access to all 61 paid skills with up to 10 cookbooks, and Pro+ (\$100/month) scales to 200 cookbooks for integrators who deploy agents at client scale.
- A complete tailor-and-deploy loop: fork a public skill, version it privately, attach it to a cookbook, deploy it — entirely through MCP tool calls, without leaving the agent conversation.

- A feedback routing system where agent field feedback lands as GitHub issues in *whichever repo the integrator configures* — not necessarily ours.

What it is not:

- A horizontal marketplace for everyone. The catalog is curated and vertical.
- A one-time purchase platform. Subscriptions only — recurring, cancel anytime.
- Free-tier first. There is one permanently free skill (`super-memory`). The value proposition is the paid catalog.
- A prompt library or RAG store. Skills are installable tools with `SKILL.md` specifications, allowlists, and versioned tarballs — not text snippets.

3. Architecture

The backend is FastAPI, reverse-proxied by Caddy. PostgreSQL is the source of truth for skills, users, subscriptions, cookbooks, forks, and feedback routing. The MCP server (`app/mcp/server.py`) runs as a StreamableHTTP endpoint alongside the REST API.

The SKILL.md contract. Every skill is a directory with a `SKILL.md` file. Frontmatter carries `slug`, `version`, `tier`, `allowlist`, and `requiredEnv`. The allowlist is the security boundary: a skill can only call the domains it declares. Agent hosts validate the allowlist before installing. The publish pipeline enforces that the declared allowlist matches actual network calls in sandboxed execution.

SkillSpector security wall. All skill content PRs run through NVIDIA SkillSpector (Apache-2.0) in CI — a static scanner with 64 vulnerability patterns across 16 categories (prompt injection, data exfiltration, privilege escalation, supply chain, MCP poisoning, and more). It runs in `--no-llm` static-only mode. Findings are emitted as SARIF to the GitHub Security tab. Advisory by default; one environment variable (`SKILLSPECTOR_BLOCK_ON_HIGH=true`) switches it to a CI blocker. Source: `docs/security/skillspector.md`.

ed25519 signing. Every published skill artifact is signed. The Recipes CLI verifies the signature before writing anything to disk. A skill whose signature does not verify is refused, even if it comes from `recipes.wisechef.ai`.

Cookbook data model. A cookbook is a named collection of skills owned by a user (`cookbook_owner` column, `CHECK ck_cookbooks_owner_required` DB constraint enforces NOT NULL). A cookbook can hold:

- Catalog skills from the public catalog (synced, auto-updated)
- Tailored private skills (promoted from the integrator's own forks)
- Pinned skills (fixed at a specific version, opt-out of auto-sync)

Cookbooks are visualizable at `/cookbooks/view?id=` — a web graph of skills as nodes, lineage arrows, and badges for pinned/tailored/corrections-absorbed.

4. The MCP tool surface

Recipes exposes 28 MCP tools via StreamableHTTP at `GET /api/mcp/http/`. Verified 2026-06-02 from `GET /skill`. Grouped by workflow:

Discovery & install

Tool	Purpose
<code>recipes_search</code>	Full-text + semantic search across the catalog
<code>recipes_recall</code>	Hybrid BM25 + vector recall ranked for your tier
<code>recipes_carousel_today</code>	Today's curated carousel
<code>recipes_install</code>	Return signed tarball URL + manifest for a slug
<code>recipes_cookbook_install</code>	Install all skills from a cookbook (bulk or single slug)

Cookbook management

Tool	Purpose
<code>recipes_list_cookbook</code>	List your cookbook and skill provenance rows
<code>recipes_sync</code>	Sync a cookbook's skills to latest versions (dry_run or apply)
<code>recipes_recipify</code>	Convert a SKILL.md draft into a CookbookSkill row

Tailoring & forks (the deploy loop)

Tool	Purpose
<code>recipes_tailor</code>	Fork a public skill to create a private editable copy
<code>recipes_fork_list</code>	List your existing forks
<code>recipes_tailor_version</code>	Upload a new version tarball to a fork (base64-encoded, MCP-native)
<code>recipes_cookbook_attach</code>	Deploy a fork's latest version into a cookbook as a private catalog skill

Cookbook handoff

Tool	Purpose
<code>recipes_cookbook_handoff</code>	Transfer or fork a cookbook to a new owner, preserving tailored skills + lineage

Publishing

Tool	Purpose
<code>recipes_publish_request</code>	Submit a skill for public catalog review

Tool	Purpose
<code>recipes_propose_skill_patch</code>	Submit a patch PR to a marketplace skill

Diagnostics

Tool	Purpose
<code>recipes_doctor</code>	Audit a local install directory for missing files and hardcoded paths
<code>recipes_seeker</code>	Probe local vendor skill directories and diff against the catalog

Community & feedback

Tool	Purpose
<code>recipes_feedback</code>	Send feedback — routes to your own repo if configured
<code>recipes_configure_feedback</code>	Configure your feedback routing target (Pro/Pro+)
<code>recipes_request_recipe</code>	Request a new skill
<code>recipes_report_skill_error</code>	Report a broken skill

Share tokens

Tool	Purpose
<code>recipes_share_create</code>	Create a cookbook share token (shown once)
<code>recipes_share_list</code>	List share tokens (metadata only)
<code>recipes_share_revoke</code>	Revoke a share token
<code>recipes_share_rotate</code>	Rotate a share token

Fleet management

Tool	Purpose
<code>recipes_fleet_create</code>	Create a named fleet of agents
<code>recipes_fleet_list</code>	List fleets and cookbook subscriptions
<code>recipes_fleet_subscribe</code>	Subscribe a cookbook to a fleet
<code>recipes_fleet_sync</code>	Sync all cookbooks in a fleet

5. The subscription model

Pricing (live as of 2026-06-02, verified against `GET /api/marketing/snapshot`):

- **Pro — \$20/month.** Every paid skill in the catalog (61 today, growing weekly). Up to 10 cookbooks. Fleet sync. Cross-vendor install (Claude Code, Cursor, Cline, OpenClaw, Hermes, Windsurf). Recurring, cancel anytime.
- **Pro+ — \$100/month.** Everything in Pro. Up to 200 cookbooks. Per-cookbook scoped API keys (up to 20). Deploy cookbooks to client agents. Private org-only catalog. Priority skill review.
- **Free.** One permanently free skill (`super-memory`). No credit card required. No time limit.

There is no founding tier, no lifetime purchase, and no per-skill charge. The DB identifiers are `cook` (Pro) and `operator` (Pro+) — stable, not changed by display label updates. The portal UI shows "Pro" and "Pro+".

DB tier identifiers vs. display labels. The API returns `tier: "cook"` and `tier: "operator"` — these are stable. The UI maps them to "Pro" and "Pro+" respectively. Any future label change touches only the translation layer. API consumers should key on `cook` and `operator` .

6. The compounding cookbook

The cookbook is the integrator's primary deliverable. Not a skill, not a prompt — the cookbook. It is what you build, tailor, hand off, and iterate on.

A cookbook starts as a named collection of catalog skills. Over time the integrator tailors some of those skills (forks + versions a private copy), attaches the tailored versions back to the cookbook, and the cookbook becomes a bespoke, versioned deployment unit that installs identically to any catalog skill but contains the integrator's own adaptations.

The compounding effect: every feedback signal the integrator receives improves the tailored skill. Every improved skill makes the cookbook more valuable to the client. Every improved cookbook, if handed off to a new client, carries the accumulated improvements. The cookbook is the moat as a product artifact.

Cookbook web visualization. Live at `/cookbooks/view?id=` . Renders skills as nodes, lineage as arrows, and badges for pinned, tailored, and corrections-absorbed count. Backend: `GET /api/cookbooks/{id}` . Verified live 2026-06-02.

Cookbook handoff. `recipes_cookbook_handoff` supports two modes:

- `transfer` — in-place ownership swap. The cookbook UUID is unchanged. Original owner loses access; new owner gains it.
- `fork` — creates a new cookbook for the new owner with `parent_cookbook_id` and `synced_from_cookbook_id` set to the source. Custom-added (tailored) skills are copied; catalog-sync rows are not.

Both modes preserve the lineage graph. A handed-off cookbook retains attribution to its tailored skills. Live, Phase I, 2026-06-02.

7. The tailor → deploy → feedback loop (the operational loop)

This is the core workflow for the integrator. Every step is an MCP tool call. No REST API knowledge required.

```

recipes_tailor          (fork a public catalog skill)
    ↓
recipes_tailor_version  (upload a modified tarball, base64-encoded)
    ↓
recipes_cookbook_attach (deploy into a cookbook → promotes to private catalog
skill)
    ↓
recipes_cookbook_install (install from the cookbook – byte-identical to a catalog
install)
    ↓
[agent runs in production, generates feedback events]
    ↓
recipes_feedback        (routes to your own repo if configured, otherwise to ours)

```

Step 1 — `recipes_tailor(source_slug, name)`

Creates a private `SkillFork` of the specified public skill. Returns `fork_id` and `fork_slug`. Idempotent: calling again with the same `(user, source_slug)` returns the existing fork. Requires Pro tier. Source: `app/mcp/tools/tailor.py`.

Step 2 — `recipes_tailor_version(fork_id, tarball_base64, semver)`

Uploads a version of the fork's modified content. The tarball is base64-encoded because MCP cannot carry multipart file uploads. Validates semver format and size (max 10 MB). Mints a `ForkVersion` row and advances `fork.latest_version_id`. Returns `version_id`, `semver`, `checksum_sha256`. Source: `app/mcp/tools/fork_deploy.py`.

Step 3 — `recipes_cookbook_attach(fork_id, target_cookbook_id)`

The bridge between the fork tables and the catalog. Reads the fork's latest `ForkVersion` tarball, extracts its `SKILL.md`, and promotes it to:

- A real `Skill` row (`is_public=False`, private to the integrator's catalog)
- A `CookbookSkill` link in the target cookbook
- A `SkillVersion` from the same tarball, minted with the canonical `recipes-skill-install` salt

This last point is the key: because the promoted unit is a real `Skill` row with a canonical install salt, `recipes_cookbook_install` installs it byte-identically to any public catalog skill — no special deploy path, no schema migration. Source: `app/mcp/tools/fork_deploy.py`.

Step 4 — `recipes_cookbook_install(cookbook_id)` (or single slug)

Installs all skills in the cookbook (or a single skill by slug). Works identically whether the skill is a public catalog skill or a tailored private skill promoted via step 3. The integration is seamless and complete.

Step 5 — feedback

After the agent runs, `recipes_feedback(category, message)` submits field feedback. Where it lands depends on routing (§8).

Dogfood verification. This loop was verified end-to-end with a real Pro account on 2026-06-02. A tailored private fork installed byte-identically to a catalog skill via `recipes_cookbook_install`. Source: Phase C integration test, `tests/test_fork_deploy.py`.

8. The feedback moat: two paths

This is the architectural feature that creates integrator lock-in — not platform lock-in, but *data-gravity lock-in in the integrator's own infrastructure*.

Default path (→ wisechef-ai/recipes-api)

Without configuration, `recipes_feedback` dispatches a GitHub issue to `wisechef-ai/recipes-api`. This is the standard path for skill improvement contributions — bug reports, feature requests, UX friction. Any Recipes subscriber gets this out of the box.

Custom path (→ your repo)

A Pro or Pro+ subscriber can configure their cookbook's feedback routing to point to *any GitHub repo they own*:

```
recipes_configure_feedback(
    repo="your-org/your-repo",
    mode="pat",
    pat="ghp_<fine-grained-PAT-with-issues:write>"
)
```

After this call, every `recipes_feedback` invocation from that subscriber's agent — for that cookbook — creates a GitHub issue *in their repo*, not ours. The issue contains the full feedback body, category, and submission ID.

Why this is the moat: The integrator's accumulated feedback lives in their own repo. Their skill improvement history is in their own repo. Their agent's field knowledge — the corpus of "here is what broke in production" — accretes in infrastructure they own and control. Recipes provides the dispatch mechanism; the data never leaves their control. This is why the loop compounds: improved skills + owned feedback history = a cookbook that gets demonstrably better over time without platform dependency.

Implementation. `recipes_configure_feedback` stores `feedback_repo`, `feedback_mode`, and an encrypted PAT (`feedback_pat_enc`) on the `Cookbook` row. The PAT is verified against the target repo before storage (GitHub API call to confirm `issues:write` permission). At dispatch time, `recipes_feedback` resolves the target by looking up the caller's cookbook for a configured `feedback_repo`. If found with `mode="pat"`, it decrypts the PAT in-memory (never logged), creates the issue via `github_dispatch.dispatch_issue`, and falls back to the default path on any failure. Source: `app/mcp/tools/feedback.py`, `app/mcp/tools/configure_feedback.py`.

Verified live. A real GitHub issue was created in a non-wisechef repo via this path on 2026-06-02. Source: Phase J integration test.

9. Trust model and security

Two-stage publishing. Any GitHub account can submit a skill PR. That PR enters as a *draft* — visible with a `draft` badge, installable by opt-in testers, not surfaced in default search. A human reviewer approves before catalog promotion.

Five-step quality pipeline (runs on every PR and every update):

- **Security scan** — SkillSpector static analysis, 64 patterns, 16 categories. SARIF output to GitHub Security tab.
- **Discipline check** — sandboxed execution with network traffic logged. Allowlist must match actual calls.
- **Quality score** — documentation completeness, test coverage, error handling. Below-threshold skills are not promoted.
- **Allowlist validation** — every domain must be resolvable and match the skill's stated purpose.
- **Manifest integrity** — frontmatter must parse cleanly, required fields present, version follows semver.

ed25519 signing. All promoted artifacts are signed. The public key is pinned in the Recipes CLI. The `install` command verifies the signature before any disk write.

Cookbook ownership enforcement. The `cookbooks.cookbook_owner` column has a `CHECK ck_cookbooks_owner_required` constraint (NOT NULL). `recipes_recipify` creates cookbooks owned by the calling user (`ctx.user_id`, fail-closed). Verified via `app/cookbook_routes.py` and live DB constraint. Source: migration `1c3005_x_cookbook_owner_ck`.

Fail-closed philosophy. Every auth predicate in the system fails closed: no `user_id` = no fork, no cookbook, no feedback routing. `authz.can_write_cookbook` returns `False` for any ambiguous state. `_require_pro_user` rejects master-key callers (no `user_id` to own anything). `_resolve_feedback_target` returns `(None, None, None)` — default path — on any lookup failure.

10. For the AI agent: reconstructing the loop from scratch

This section exists for the AI agent reader. If you are an AI agent ingesting this document to learn how to operate the Recipes platform, this is the minimum viable operational sequence.

Pre-requisite: You have a Pro-tier API key (`RECIPES_API_KEY=rec_*`). Your MCP client is configured with `RECIPES_API_KEY` in the environment.

Install a skill from the catalog:

```
recipes_search(query="<what you need>")      # find the slug
recipes_install(slug="<slug>")               # get signed tarball URL
# download tarball, verify ed25519 signature, extract to skills/
```

Tailor a skill for private use (the full loop):

```
# 1. Fork
recipes_tailor(source_slug="<public-slug>", name="my-tailored-version")
# → {fork_id: "...", fork_slug: "..."}

# 2. Version (package your modified SKILL.md + scripts as a .tar.gz, base64-encode it)
recipes_tailor_version(fork_id="...", tarball_base64="<base64>", semver="1.0.0")
# → {version_id: "...", checksum_sha256: "..."}

# 3. Attach to cookbook (get cookbook_id from recipes_list_cookbook)
recipes_cookbook_attach(fork_id="...", target_cookbook_id="...")
# → {skill_slug: "...", version: "1.0.0", is_public: false}

# 4. Install
recipes_cookbook_install(cookbook_id="...")
# installs all skills in the cookbook, including your tailored one
```

Configure feedback routing to your own repo:

```
recipes_configure_feedback(
    repo="your-org/your-repo",
    mode="pat",
    pat="ghp_<issues-write-token>"
)
# Verify: call recipes_feedback – issue should appear in your repo
```

The two feedback paths, answered directly:

- Path A (default): `recipes_feedback` → `github_dispatch.dispatch_event` → issue in `wisechef-ai/recipes-api`. No config needed.
- Path B (the moat): `recipes_configure_feedback(repo=..., mode="pat", pat=...)` → `recipes_feedback` → issue in *your repo*. Requires Pro/Pro+. Your feedback corpus stays in your infrastructure.

11. Get started

Three entry points:

Evaluate: Install `super-memory` free at recipes.wisechef.ai/skills/super-memory. No account required. Demonstrates the full install UX — allowlist validation, signature verification, agent host handshake — in under 60 seconds.

Subscribe: Browse the catalog at </skills>. Filter by category, tier, and agent host. Pricing at </pricing>. Install docs for Claude Code, Cursor, Cline, OpenClaw, Hermes, and Windsurf at </docs/install>.

Tailor and deploy: Once subscribed (Pro), the tailor loop is live. Use `recipes_tailor` → `recipes_tailor_version` → `recipes_cookbook_attach` → `recipes_cookbook_install`. Configure feedback routing with `recipes_configure_feedback`. Your compounding cookbook starts with the first `recipes_cookbook_attach` call.

Appendix: verified claims

All quantitative claims in this document are sourced. The complete claim ledger is at `docs/WHITEPAPER_CLAIMS.md` in the `wisechef-ai/recipes-portal` repository.

Claim	Source	Verified
62 total skills	<code>GET /api/marketing/snapshot</code> → <code>counts.skills_total</code>	2026-06-02
1 free skill	<code>GET /api/marketing/snapshot</code> → <code>counts.free_skills</code>	2026-06-02
61 Pro skills	<code>GET /api/marketing/snapshot</code> → <code>counts.pro_skills</code>	2026-06-02
28 MCP tools	<code>GET /skill grep of recipes_* tool names</code>	2026-06-02
Pro \$20/month	<code>GET /api/marketing/snapshot</code> → <code>tiers.pro.price_usd</code>	2026-06-02
Pro+ \$100/month	<code>GET /api/marketing/snapshot</code> → <code>tiers.pro_plus.price_usd</code>	2026-06-02

Claim	Source	Verified
Pro: 10 cookbooks	GET /api/marketing/snapshot → counts.pro_cookbooks	2026-06-02
Pro+: 200 cookbooks	GET /api/marketing/snapshot → counts.pro_plus_cookbooks	2026-06-02
Tailor loop complete + byte-identical	app/mcp/tools/fork_deploy.py + Phase C test	2026-06-02
Feedback moat live	app/mcp/tools/feedback.py + Phase J test	2026-06-02
Cookbook viz live	/cookbooks/view?id= + GET /api/cookbooks/{id}	2026-06-02
Cookbook ownership constraint	Migration lc3005_x_cookbook_owner_ck, CHECK ck_cookbooks_owner_required	2026-06-02
SkillSpector advisory CI	docs/security/skillspector.md, NVIDIA/skillspector Apache-2.0	2026-06-02
Cookbook handoff live	app/mcp/tools/cookbook_handoff.py, Phase I	2026-06-02